INSTITUTO POLITÉCNICO
UNIVERSIDADE KIMPA VITA
DEPARTAMENTO DE ENGENHARIA INFORMÁTICA

LINGUAGEM DE PROGRAMAÇÃO Python

Índices Baseados em Hashing

Elementos do Grupo :

1- Chiminha André Mufua
2- Dinie Raúl Silvestre
3- Mankenda Vicente
4- Nelson Edgar Tonoca
5- Rosário Canga

Grupo n.:

Ano: 3º

Turno: Laboral

Docente Moyo Kanivengidio

Uíge/2026

Conteúdo

1 Introdução 3
1 Conceitos de Hashing 4
1.1 Estrutura de um índice hash 4
1.2 Tipos de hashing 5
1.2.1 Hashing Estático 5
1.2.2 Hashing Dinâmico 6
1.2.3 Hashing Extensível 6
1.2.4 Hashing Linear 6
2 Tratamentos de colisões 6
2.1 Encadeamento (Chaining) 6
2.2 Endereçamento Aberto 7
2.3 Rehashing 7
2.4 Complexidade 7
3 Comparação com outros índices 7
4 Aplicação prática 7
4.1 Bancos de dados 7
4.2 Segurança 8
4.3 Sistemas de cache 8
4.4 Estruturas de dados 8
4.5 Vantagens 8
4.6 Desvantagens 8
5 Conclusão 9
6 Referências 10

1 Introdução

Na área de Ciência da Computação, a organização eficiente de dados é essencial para garantir rapidez e desempenho em sistemas.

Um dos métodos mais eficientes para isso é o índice baseado em hashing, muito usado em bancos de dados para permitir acesso quase imediato às informações.

Diferente de outros métodos de indexação, como árvores, o hashing não precisa percorrer estruturas complexas ele vai direto ao ponto.

1 Conceitos de Hashing

Ás vezes, nós queremos recuperar os registros de uma tabela especificando os valores de um ou mais campos · Encontrar todos os estudantes cujo curso é ’BCC’ · Encontrar todos os estudantes cujo CR > 7 Um Índice recupera os registros de forma rápida. através de uma chave de pesquisa · Qualquer subconjunto dos campos de uma relação podem ser uma chave de pesquisa · Chave de Pesquisa não é uma chave primária, pois não precisa necessariamente ser única

Hashing é uma técnica que transforma uma chave de entrada em um valor fixo por meio de uma função matemática. Esse valor é chamado de código hash.

Características de uma função hash:

- ✓ Rápida de calcular
- ✓ Distribui bem os dados
- ✓ Evita colisões
- ✓ Determinística (mesma entrada → mesma saída)

1.1 Estrutura de um índice hash

Índice - Coleção de entradas de dados que suporta a recuperação eficiente de registros combinando em uma certa condição de pesquisa. Seletividade · Medida da quantidade de variedade existente nos valores de uma dada coluna em relação ao número total de linhas de uma dada tabela.

Índices podem ser:

- ✓ Agrupados (Clustered)
- ✓ Desagrupados (Unclustered)
- ✓ Denso ou Esparso
- ✓ Primário ou Secundário

Agrupado (Clustered) - Se a ordem dos registros e dos índices é a mesma ou próxima

Desagrupado (Unclustered) · Não existe ordem

Esparso - Um dado de entrada para cada página de arquivo · Usa menos espaço de armazenamento, porém leva mais tempo para localizar um registro dada a sua chave Denso ·

Pelo menos um dado de entrada por chave-valor · Seqüência de blocos contendo apenas as chaves dos registros e os ponteiros para os próprios registros

Um índice baseado em hashing é composto por:

a) Chave (Key)

Valor usado para identificar um registro

Ex: número do aluno, ID, email

b) Função Hash

Transforma a chave em um índice

c) Buckets (baldes)

Locais onde os dados são armazenados . Cada bucket pode conter:

Um registro

Ou vários (em caso de colisão)

1.2 Tipos de hashing

Existem diferentes formas de implementar hashing:

1.2.1 Hashing Estático

Transformar cada chave de busca de 0, 1, ..., N em um intervalo de 0, 1, ..., M · Utilizando uma função hash(key) · Podem ser Clustered ou Unclustered · Tempo médio de busca O(1)

Características

A função hash mapeia uma chave para um determinado bucket · No pior caso ela mapeia várias entradas para o mesmo bucket · No melhor caso ela mapeia todo valor chave-pesquisa para um bucket diferente · Funções hash devem ter distribuição uniforme e aleatória

- ✓ Número fixo de buckets
- ✓ Simples de implementar
- ✓ Pode gerar muitas colisões com o tempo

1.2.2 Hashing Dinâmico

A função hash é modificada dinamicamente Hash Extensível (Extendible Hashing) · Calcula o hash de cada chave para uma string de bits longa, mantendo o tamanho de acordo com o necessário · Mantém um diretório com ponteiros para os buckets do hash · A função hash gera um valor dentro de um intervalo, geralmente um b-bit = 32

Operações

- Insertion - O endereço do bucket é calculado · Caso o bucket esteja cheio deve-se adicionar mais um bucket e adicionar mais um bit para o hash · Caso contrário, inserir o dado no bucket
- Search - Verificar a profundidade do prefixo e usar os bits para encontrar o endereço do bucket após o hash
- Delete - Uma operação de busca seguida por uma de deleção do dado

Adapta-se ao crescimento dos dados

1.2.3 Hashing Extensível

- ✓ Usa diretórios
- ✓ Cresce conforme necessário

1.2.4 Hashing Linear

- ✓ Expande gradualmente
- ✓ Mais eficiente em sistemas grandes

2 Tratamentos de colisões

Colisão ocorre quando duas chaves diferentes geram o mesmo índice.

Principais técnicas:

2.1 Encadeamento (Chaining)

- ✓ Cada posição aponta para uma lista de elementos
- ✓ Fácil de implementar

2.2 Endereçamento Aberto

- ✓ Procura outra posição livre

Tipos:

- ✓ Linear
- ✓ Quadrático
- ✓ Duplo hashing

2.3 Rehashing

- ✓ Aplica uma nova função hash
- ✓ Redistribui os dados

2.4 Complexidade

O hashing é conhecido pela sua eficiência:

Melhor caso: O(1) (tempo constante)

Caso médio: O(1)

Pior caso: O(n), quando há muitas colisões

Isso o torna muito mais rápido que buscas sequenciais.

3 Comparação com outros índices

Vamos comparar com árvores (ex: B-tree):

CaracterísticaHashingÁrvoresBusca exataMuito rápidaRápidaBusca por intervaloRuimExcelenteComplexidadeO(1)O(log n)Uso comumIgualdade (=)Intervalos.

4 Aplicação prática

O índice hash é amplamente utilizado em:

4.1 Bancos de dados

Sistemas como MySQL e PostgreSQL usam hashing para acelerar consultas.

4.2 Segurança

- ✓ Na área de Segurança da Informação:
- ✓ Senhas são armazenadas como hash
- ✓ Protege contra roubo de dados

4.3 Sistemas de cache

1. Armazenamento rápido de dados temporários

Ex: páginas web

4.4 Estruturas de dados

- ✓ Tabelas hash
- ✓ Dicionários em linguagens de programação

4.5 Vantagens

a) Muito rápido
b) Fácil acesso direto
c) Ideal para grandes volumes de dados.
d) Simples de implementar.

4.6 Desvantagens

a) Não permite ordenação
b) Ineficiente para intervalos

5 Conclusão

O índice baseado em hashing é uma das técnicas mais rápidas para acesso a dados.

Mesmo com desafios como colisões, ele continua sendo essencial em sistemas modernos devido à sua eficiência e simplicidade.

6 Referências

Documentação do MySQL

Sadalage, P. J., & Fowler, M. (2012). NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence. Addison-Wesley.

MongoDB Documentation. (n.d.). MongoDB Manual. Retrieved from https://docs.mongodb.com/manual/

Documentação do PostgreSQL

Livros de Banco de Dados